Distributed Task Scheduling for MEC-Assisted Virtual Reality: A Fully-Cooperative Multiagent Perspective

Ziqi Li, Heli Zhang, Xi Li, *Member, IEEE*, Hong Ji, *Senior Member, IEEE*, and Victor C.M. Leung, *Life Fellow, IEEE*

***Abstract*—In the upcoming 6th generation of mobile networks, wireless virtual reality will become an essential application. Due to the heterogeneity of rendering requirements and edge servers, how to provide adaptable task scheduling capabilities in edge networks becomes an important issue. However, most related studies regard edge servers as self-interested nodes that make scheduling decisions independently, ignoring the potential power of cooperation among edge servers. In this article, we propose a fully-cooperative task scheduling scheme for virtual reality, taking into account parallel multitasking, the randomness of task arrival and leaving, as well as network load balance. By sharing a common objective among distributed schedulers, tasks can be offloaded to proper edge servers in a cooperative way. We first formulate an optimization problem to maximize the number of delay-satisfied tasks in the entire edge network. Next, we treat the edge network as a fully-cooperative multi-agent system and transform the problem into a decentralized partially observed Markov decision process (Dec-POMDP). Finally, we propose a cooperative task scheduling scheme based on multi-agent proximal policy optimization to solve the Dec-POMDP. Simulation results show that the cooperation among edge nodes brings the network improvement in terms of offloading success rate, delay compliance rate, and load balance.**

***Index Terms*—6G, virtual reality, multi-access edge computing, distributed collaboration, multi-agent PPO.**

I. Introduction

RECENT years have witnessed the rapid growth of immersive virtual reality (VR). To support the rendering of high-definition frames, today's high-end VR devices (such as Oculus Rift and HTC Vive) are usually connected to a powerful server via an HDMI interface [1]. In the approaching 6G era, with the peak data rate of 1000 Gbps, the seamless streaming of wireless VR would be possible [2], which allows users to slip the bonds of cable and meet the truly immersive experience. By 2030, it is expected that the market size for wireless VR will reach $44.7 billion [3].

In order to provide wireless VR devices with powerful computing resources, multi-access edge computing (MEC) -assisted VR rendering has become a promising framework [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]. In this framework, a part of the rendering workload is offloaded from VR devices to a nearby MEC server that has more computing and energy resources. However, considering the randomness of user access and the suddenness of VR tasks, a single MEC server may face the ups and downs of its workload. Besides, due to the heterogeneous task requirements and server resources, the nearest MEC server may not meet the quality of experience (QoE) that the user desired [14].

Aiming to further exploit the edge computing power, peer-to-peer (P2P) offloading among MEC nodes is recently proposed to schedule tasks among multiple edge servers [15], [16], [17], [18], [19], [22]. In this scheme, an MEC server (**the source node**) offloads its workload to a nearby peer server (**the destination node**), aiming to better unleash the computing power of the edge network. An important observation, however, is that in these studies, the task scheduling is self-interested, meaning that each server has its own objectives and only cares about its own interests when making offloading decisions. From the perspective of the entire system, self-interest makes the edge network lack coordination, resulting in conflicting scheduling decisions and even a deadlock in allocating resources. Therefore, it is difficult for self-interested task scheduling to achieve the best user experience and load balance.

In this article, we demonstrate that cooperative scheduling of VR tasks can improve the average QoE within edge networks. The term "cooperative" refers to the fact that all scheduling nodes have the same objective: to maximize the average QoE of all tasks. To this end, there are several challenges to be addressed. First, under the edge network architecture without a central controller, it is challenging for scheduling nodes to achieve effective cooperation in decision-making. Second, given the diversity of task requirements and computing resources, finding the best match between task and server is difficult, especially in the absence of global information. Last, each server's resource

Manuscript received 12 July 2022; revised 15 June 2023; accepted 5 February 2024. Date of publication 13 February 2024; date of current version 16 July 2024. This work was supported by the National Natural Science Foundation of China under Grant 62171061. The review of this article was coordinated by Dr. Guiyi Wei. *(Corresponding author: Heli Zhang.)*

Ziqi Li, Heli Zhang, Xi Li, and Hong Ji are with the Key Laboratory of Universal Wireless Communications, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zqli@bupt.edu.cn; zhangheli@bupt.edu.cn; lixi@bupt.edu.cn; jihong@bupt.edu.cn).

Victor C.M. Leung is with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with the Department of Electrical and Computer Engineering, the University of British Columbia, Vancouver, BC V6T1Z4, Canada (e-mail: vleung@ieee.org).

Digital Object Identifier 10.1109/TVT.2024.3365476

utilization should be retained at a suitable level during task scheduling, so as to provide the edge network with a good load balance.

Tackling these challenges, the main contributions of this work are summarized as follows:

- For MEC-assisted VR, we propose a distributed fully-cooperative task scheduling scheme. In contrast to self-interested task scheduling, this scheme enables scheduling nodes to share a common objective while making decisions. Although this article focuses on the wireless VR scenario, the scheme can also be applied to other delay-sensitive services, such as real-time multimedia and the Industrial Internet of Things.
- We consider the situation where multiple tasks arrive at an edge server randomly and could be terminated by users at any time. We model the compound process of task arriving and task leaving as a Simple Death with Immigration process, then arrange computing resources using the virtual machine-based parallel multitasking.[1] A dynamic optimization problem is formulated to maximize the number of delay-satisfied tasks and ensure load balance.
- To solve the formulated problem, we treat the edge network as a multi-agent system and propose a cooperative task scheduling scheme based on the multi-agent proximal policy optimization algorithm. The simulation results suggest the superiority of the cooperative scheduling scheme, especially when the task arrival rate is moderate.

The rest of the article is organized as follows. We first briefly review the related works of this article in Section II, then give the overview and mathematical description of the system model in Section III. Section IV introduces the proposed cooperative task scheduling scheme for MEC-assisted VR. Simulation results and the discussions are shown in Section V, followed by the concluding remarks in Section VI.

II. Related Work

In this section, we first present a brief review of existing works in MEC-assisted VR. Next, we introduce recent studies on peer-to-peer offloading among MEC nodes, which is an important basis for task scheduling in edge networks.

A. MEC-Assisted Vr

Generally speaking, recent studies on MEC-assisted VR fall into the following five categories.

Content caching: In [4], the authors proposed a tile popularity prediction approach based on the estimation of the tile saliency using the Choquet integral. Based on the popularity, the tiles were collaboratively cached in multi-cell 5G MEC networks to improve the network caching performance. Also, [5] proposed a dynamic caching replacement scheme to reduce repetitive communication overhead and unnecessary resource utilization at both the MEC server and the local VR device.

Resources allocation: The work of [6] suggested MEC-assisted rendering for panoramic VR. This work demonstrated that the MEC server could be deployed near the base station (BS) as a cache server and a powerful computing unit. A communication and computation resource allocation problem for MEC server and user equipment (UE) was formulated to maximize the viewport quality, while minimizing the energy consumption of the UE. The authors of [7] studied the resource management problem in MEC-assisted VR games, in which association, caching policy, and offloading mode selection were jointly optimized to maximize users' QoE utility. In [8], the authors suggested that the terahertz wireless access-based MEC system should be used to support high-quality immersive VR video services. An optimization problem was formulated by jointly optimizing the binary viewport rendering offloading decision and the downlink transmission power of the MEC server to minimize the long-term averaged energy consumption.

Tile based video transmission: In [5], the authors studied tile based offloading for VR video services. Instead of proportional offloading, this article proposed multi-tile deterministic offloading, where the field of view (FoV) tasks were segmented into tiles and the basic allocation unit is one single tile of a frame.

Viewpoint prediction: In article [9], the authors proposed an MEC-enabled wireless VR network, in which the FoV of each VR user could be real-time predicted using recurrent neural networks. Based on the predictions, the rendering of FoV content was moved from VR device to MEC server with rendering model migration capability.

User association: Aiming to minimize the occurrence of breaks in presence (BIP) that could detach the users from their virtual world, the authors of [10] formulated user association policy as an optimization problem, considering the predictions of users' locations, orientations, and their BS association. Next, they solved the problem using a distributed learning algorithm based on deep echo state networks.

B. Peer-to-Peer Offloading Among MEC Nodes

The work of [15] investigated computation peer offloading for energy-constrained MEC in small-cell networks, which is an early effort on peer-to-peer offloading among MEC nodes. In this work, the authors proposed both centralized and decentralized solutions. In the latter solution, the problem was formulated as a non-cooperative game, in which small-cell base stations manage to minimize their own costs in a decentralized and autonomous manner. In [16], the authors demonstrated that trust is an important issue in peer-to-peer offloading. A scheme called OLCD was proposed to predict the trust values of all peer nodes. By implementing trust propagation along multi-hop paths, this work made it possible to achieve trusted mobile edge computing. The shortcoming of this approach is that the time spent on trust propagation before it converges is unignorable, especially when the scale of the edge network is large. In [17], the authors proposed a peer-to-peer offloading scheme for pervasive edge computing based on generalized adversarial imitation learning (GAIL). Although GAIL is a promising approach to dealing with complicated offloading decisions, the acquisition of expert policies is quite a challenge in practical scenarios.

The authors of [18] and [19] investigated the peer-to-peer offloading scheme for edge computing, considering both

[1]In this article, parallel multitasking means that rendering tasks generated by different users can be processed simultaneously on an edge server.

multiple tasks as well as multi-hop paths. The highlight of these two works is that they considered the flow scheduling of the network. The weakness is that the proposed schemes are not decentralized because of the existence of central controllers. Similarly, the work in [20] and [21] studied task scheduling and collaborative computing respectively, but both with the need for a central controller. In [22], the authors formulated the peer offloading problem based on the stochastic arrival model. Two algorithms were proposed to maximize the utility function of throughput with the constraint of time-average energy consumption and worst-case response time. However, this work supposed that tasks could be done within a time slot and did not consider multiple heterogeneous tasks running in parallel. The work of [23] studied distributed task offloading in edge networks from a self-interested perspective, where each edge server makes scheduling decisions independently. In [24], the authors proposed an online task scheduling scheme for edge networks. This scheme supports distributed execution but it requires frequent message exchange among edge nodes during the execution phase, bringing a large communication cost and long decision time.

Recently, a joint parallel offloading and load balancing policy is proposed under the software-defined network architecture [25]. It enables tasks to be computed simultaneously by end devices, edge servers, and a remote cloud. The works of [26] and [27] address the problem of peer offloading among servers based on Lyapunov optimization and queuing theory. In [28], edge servers are segmented into groups, and the average delay of each group is optimized using a centralized approach, while the total revenue of each group is maximized using a non-cooperative game-based scheme. Additionally, in [29], a trust and incentive mechanism for MEC peer offloading is proposed with the help of Blockchain.

Note that most related studies on peer offloading either adopt non-cooperative decisions or employ centralized methods. The investigation of distributed cooperative decision-making in peer offloading remains limited. To the best of our knowledge, the work most similar to ours is [30], which presents a game-based cooperative computation offloading method for vehicular edge computing networks. However, this method requires user vehicles to possess resource information about servers and overlooks the scenario of multiple tasks being processed simultaneously by a server. Therefore, as far as we know, there are currently no existing schemes that can be directly applied in the specific scenario considered in this article.

III. System Model and Problem Formulation

In this section, we first describe the overview and the modeling of the system, then provide the detail of the problem formulation.

A. System Overview

In the considered scenario of this article, the rendering of VR content has six steps, as is shown in Fig. 1. By repeating the fifth and sixth steps, the user can obtain wireless VR rendering services with the assistance of the edge server. It should be noted that the specific offloading mechanism between the VR device and the MEC server [4], [7], [11] is not the main concern of this article.

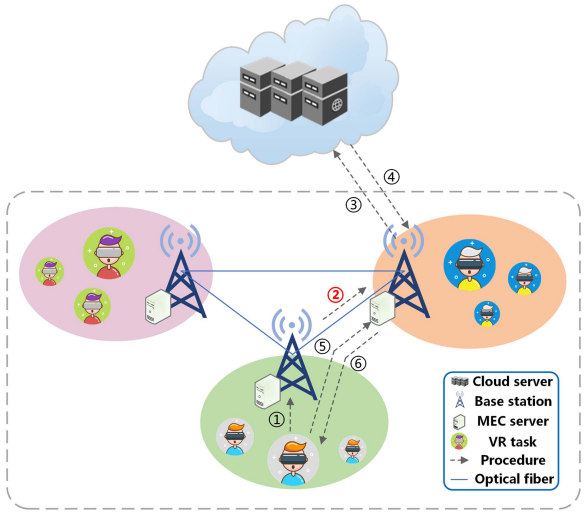

Fig. 1. Considered scenario of MEC-assisted wireless VR. There are six steps in the rendering of VR content: 1) User requests rendering service from source BS. 2) The source BS decides where to render a task by choosing a destination BS, i.e. the task scheduling. 3) The destination BS requests rendering materials from the cloud server. 4) Download rendering materials. 5) The user sends interactive instructions to the destination BS. 6) The destination BS provides content rendering and sends back frames.

The main focus of this article is on the cooperative task scheduling of the entire edge network. In this article, **cooperation refers not only to offloading a task from one server to another, but also to maintaining the coordination of decision-making in the process of selecting destination servers**. In the considered scenario, each base station acts as both a scheduler and an executor.[2] On the one hand, a base station is responsible for scheduling tasks to appropriate computing nodes. On the other hand, the base station itself can also serve as a computing node for rendering. For the entire edge network, the evolution of its state (task assignment, node workload) is determined by the joint decision of all schedulers. To achieve cooperation, a scheduler needs to take into account and interact with not only the network but also other schedulers.

Fig. 2 provides an example of the distributed cooperation among MEC servers, in which there are three BSs (BS 1, BS 2, and BS 3) and four active VR tasks (Task 1, Task 2, Task 3, and Task 4). With the consideration of other BSs' actions, BS 2 decides to serve Task 3 locally; BS 1 decides to serve Task 2 locally but offload Task 1 to BS 3; BS 3 decides to offload Task 4 to BS 2. The decision-making in the example is fully-decentralized. A mathematical description of the considered system is provided below, with the important notations summarized in Table I.

B. System Modeling

As is shown in Fig. 2, consider a cellular network that has M BSs, each of which is equipped with an MEC server. Let

[2]Each base station is treated as a scheduler and an executor for simplicity. In practice, the functions of the scheduler and the executor may be assigned to different physical entities.

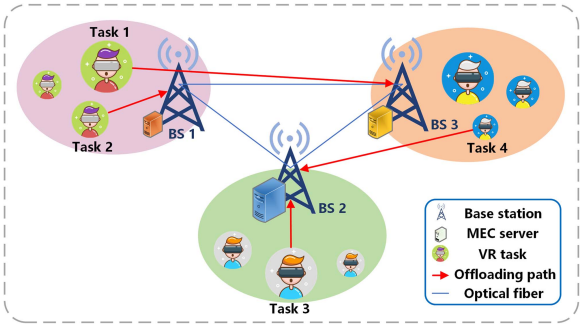

Fig. 2. Distributed cooperation among MEC servers.

TABLE I
NOTATIONS

Symbol	Description
$\mathcal{M}$	Set of BSs or MEC servers
M	Number of BSs or MEC servers
m	A certain BS or MEC server
$\mathcal{U}_t$	Set of users or VR tasks in the edge network
$\mathcal{U}_t^m$	Set of users or VR tasks under BS m
U_t^m	Number of users or VR tasks under BS m
u_t^m	A certain user or VR task under BS m in time slot t
$s_{u_t^m}$	Frame's data size of VR task u_t^m
$r_{u_t^m}$	Required CPU cycle to render a frame of task u_t^m
p_m	Total computing capability of MEC server m
C_t^m	Number of occupied VMs of BS m in time slot t
L_t^m	Number of leaving tasks of BS m in time slot t
I_m	Limit on VMs of MEC server m
d	Capacity of optical fiber
Ψ	Number of arrived tasks
Θ	Number of existing tasks
t	Timestamp
$\mathcal{T}$	Set of time slots
T	Limit of time slot or time step
$\mu, \sigma, \epsilon, \rho$	Parameters of Gaussian distribution
λ	Rate factor of task arrival process
β_l	Rate factor of task leaving process
β	Basic rate parameter of task leaving process
$\mathcal{N}$	Non-negative integer set
$f_{mm'}^{u_t^m}$	Decision variable related to P2P offloading
$\sigma_{mm'}$	A 0-1 variable related to transmission delay
T_{thr}	Maximum tolerance delay of VR tasks

$\mathcal{T} = \{0, 1, \ldots, t, \ldots, T\}$ be the set of time slots. The set of all BSs is denoted as $\mathcal{M} = \{1, 2, \ldots, m, \ldots, M\}$, serving a set $\mathcal{U}_t$ of wireless VR users in time slot t. In this article, each user is corresponding to a VR task, and the term user is interchangeable with the term task. Under the BS m in time slot t, there are U_t^m users, which can be denoted as $\mathcal{U}_t^m = \{1, 2, \ldots, u_t^m, \ldots, U_t^m\}$. $\mathcal{U}_t$ and $\mathcal{U}_t^m$ satisfy the following relationship:

$$\bigcup_{m=1}^{M} \mathcal{U}_t^m = \mathcal{U}_t. \tag{1}$$

1) Attributes of VR Tasks: Rendering is the critical process of VR services, involving building graphics from raw materials and models (including lighting, texture, shape, etc). The rendering process has two features. First, to achieve a smooth user experience, VR has a strict demand for Motion-to-Photon (MTP) latency. When the MTP latency is less than 20 ms, the feeling of dizziness due to the desynchrony of motion and display will be alleviated to a large extent [31]. Second, VR contents need a high-definition resolution display, which leads to the large size of raw graphic material.

For VR task u_t^m, the basic attributes include the size of each frame $s_{u_t^m}$ (bit) and the required CPU cycle to render a frame $r_{u_t^m}$ [7], [32]. Considering that different VR tasks have different graphic settings (such as resolution, lighting, motion blur, anti-aliasing, and so on), the attributes of VR tasks are heterogeneous. In this article, we suppose $s_{u_t^m}$ and $r_{u_t^m}$ of different tasks obey Gaussian distribution $\Phi(\mu, \sigma^2)$ and $\Phi(\epsilon, \rho^2)$ respectively. Besides, we assume that $s_{u_t^m}$ and $r_{u_t^m}$ are fixed for the same task.

2) Computing: In order to support multiple tasks running in parallel, we consider a Virtual Machine (VM) based resource virtualization mechanism [33], where each VR task can be assigned to a VM for graphics rendering. A VM can be seen as an emulation of a computer system, which uses software instead of a physical computer to run programs and deploy applications. Compared with other resource virtualization mechanisms (such as containers), VM has the advantage of providing stable hardware resources to support delay-sensitive applications. This advantage makes VM an ideal carrier for VR rendering.

We further consider the heterogeneity of each MEC server. Specifically, it can be supposed that:

- The total CPU resources (i.e. CPU frequency) of different servers are different.
- For a certain server, its total CPU resources are divided into several parts, and each of them can be allocated to a VM. Each VM can carry at most one rendering task.
- Each server has a limit on the number of supported VMs, and this limit varies from server to server.

Considering the potential intolerance of VR users towards the lengthy VM initialization time [34], [35], we adopt a proactive approach by creating VMs in advance, thus when rendering tasks arrive, they can be immediately processed without delay. Note that at this time we cannot allocate hardware resources for each VM based on task requirements, as the task attributes are totally random and unpredictable. Taking this into account, we refer to the work of [36] to evenly divide the hardware resources of a server.

For server m, let p_m (cycle/s) be its total computing capability and I_m be its limit on the number of VMs, then the time spent on rendering a frame of task u_t^m can be represented as

$$T_{\text{render}} = \frac{r_{u_t^m}}{\frac{p_m}{I_m}} = \frac{r_{u_t^m} I_m}{p_m}. \tag{2}$$

3) Transmission: The total delay in an edge network consists of queuing delay, transmission delay, and propagation delay. Considering that BSs are connected via optical fiber and the speed of light is fast enough, the propagation delay can be ignored. The queuing delay can also be ignored because a BS is usually connected to another directly, without the need for routing [37]. The main influencing factor is the transmission delay, which can be denoted as d (bit/s). Thus, we can represent the time spent on transferring a frame of task u_t^m as

$$T_{\text{trans}} = \frac{s_{u_t^m}}{d}. \tag{3}$$

There are three additional notes about transmission delay. First, the above transmission delay exists only when the source BS and the destination BS are different. Second, the transmission

delay of interactive control and head-moving instructions can be ignored because of their small data size [33]. Finally, the delay of wireless transmission between a VR device and a BS is not counted here, as the primary focus of this article is task scheduling in edge networks.

4) Task Arrival and Leaving: The arrival of VR tasks in the edge network can be modeled as a Poisson process. The number of arrived tasks Ψ obeys the Poisson distribution with the rate λ, where λ refers to the average number of arrived tasks in a time unit. Thus, at time t, random variable Ψ satisfies the following equation:

$$P\{\Psi(t)=k\}=\frac{(\lambda t)^k e^{-\lambda t}}{k!}, k\in\mathcal{N}, \tag{4}$$

where $\mathcal{N}$ refers to non-negative integers.

Considering that a user may terminate a VR application randomly, the leaving of tasks in the edge network can be regarded as an extended form of Poisson processes. Specifically, the leaving process can be modeled as a death process, where the leaving rate β_l is not a constant but is related to the current number of existing tasks in the network. Next, we modeled the compound of task arriving and leaving as a Simple Death with Immigration process. Let Θ be the random variable that describes the number of existing tasks in the network, for $k,l\in\mathcal{N}$, Θ satisfies

$$P\{\Theta(t+\triangle t)=k|\Theta(t)=l\}=\begin{cases}\lambda\triangle t, & k=l+1,\\ \beta_l\triangle t, & k=l-1,\\ o(\triangle t), & |k-l|\geq 2,\end{cases} \tag{5}$$

where $\beta_l=l\beta$, reflecting the fact that the leaving rate increases as l grows. $o(\triangle t)$ is an infinitesimal of $\triangle t$, indicating that the arrival or leaving of tasks happens no more than once during a time slot t, as long as the time slot is segmented short enough.

5) Decision Variable, Constraints, and Optimization Objective: The decision variable of the system can be denoted as

$$f_{mm'}^{u_t^m}\in\{0,1\}, \forall m\in\mathcal{M}, t\in\mathcal{T}, u_t^m\in\mathcal{U}_t^m. \tag{6}$$

This variable indicates whether to offload a task u_t^m form BS m to BS m' in time slot t. m and m' can be identical, which means that the task is computed locally in the source node. If $f_{mm'}^{u_t^m}=1$, the task u_t^m will be offloaded from BS m to BS m', otherwise not.

There are three constraints that should be satisfied while offloading a task. First, every generated VR task in the edge network should be scheduled once exactly, thus ensuring that every task can be served by a VM. This constraint can be represented as

$$\sum_{m'=1}^{M} f_{mm'}^{u_t^m}=1, \forall m\in\mathcal{M}, t\in\mathcal{T}, u_t^m\in\mathcal{U}_t^m. \tag{7}$$

The second constraint is that for any server m', the number of allocated VM cannot exceed its limit, which is a basic requirement of load balance. This constraint can be represented as

$$\sum_{m=1}^{M}\sum_{u_t^m=1}^{U_t^m} f_{mm'}^{u_t^m}+C_t^{m'}-L_t^{m'}\leq I_{m'}, \forall m'\in\mathcal{M}, t\in\mathcal{T}, \tag{8}$$

where

$$C_t^{m'}=\begin{cases}\sum_{m=1}^{M}\sum_{u_{t-1}^m=1}^{U_{t-1}^m} f_{mm'}^{u_{t-1}^m}+C_{t-1}^{m'}-L_{t-1}^{m'}, & t\geq 1,\\ 0, & t=0.\end{cases} \tag{9}$$

In (8), the first term $\sum_{m=1}^{M}\sum_{u_t^m=1}^{U_t^m} f_{mm'}^{u_t^m}$ refers to the number of tasks that are assigned to BS m'. The second term $C_t^{m'}$ refers to the number of occupied VMs of BS m' in time slot t. The definition of $C_t^{m'}$ is recursive and its value is easy to obtain for BSs. The last term $L_t^{m'}$ is the number of leaving tasks of BS m' in time slot t. Its value can be easily obtained by the BS, too.

Last, the total delay that consists of rendering delay and transmission delay cannot exceed the maximum tolerated delay T_{thr}. It can be represented as

$$f_{mm'}^{u_t^m}\left\{\underbrace{\frac{r_{u_t^m}I_{m'}}{p_{m'}}}_{T_{\text{render}}}+\underbrace{\frac{s_{u_t^m}}{d}\cdot\sigma_{mm'}}_{T_{\text{trans}}}\right\}\leq T_{\text{thr}}, \tag{10}$$

$$\forall m\in\mathcal{M}, t\in\mathcal{T}, m'\in\mathcal{M}, u_t^m\in\mathcal{U}_t^m,$$

where

$$\sigma_{mm'}=\begin{cases}1, & m\neq m',\\ 0, & m=m'.\end{cases} \tag{11}$$

The optimization objective is to maximize the number of tasks that satisfied the delay constraint and the VM limit constraint in a long term, which can be written as

$$\sum_{t=0}^{T}\sum_{m'=1}^{M}\sum_{m=1}^{M}\sum_{u_t^m=1}^{U_t^m}\mathcal{L}\left(f_{mm'}^{u_t^m}\right), \tag{12}$$

where

$$\mathcal{L}\left(f_{mm'}^{u_t^m}\right)=\begin{cases}1, & \text{if (8) and (10) are satisfied,}\\ 0, & \text{else.}\end{cases} \tag{13}$$

C. Problem Formulation

Based on equation (1)–(13), the optimization problem can be formulated as

$$\mathbf{P1}: \max_{\{f_{mm'}^{u_t^m}\}_{t\in\mathcal{T}}} \sum_{t=0}^{T}\sum_{m'=1}^{M}\sum_{m=1}^{M}\sum_{u_t^m=1}^{U_t^m}\mathcal{L}\left(f_{mm'}^{u_t^m}\right) \tag{14}$$

$$\text{s.t.} \sum_{m'=1}^{M} f_{mm'}^{u_t^m}=1 \tag{14a}$$

$$\sum_{m=1}^{M}\sum_{u_t^m=1}^{U_t^m} f_{mm'}^{u_t^m}+C_t^{m'}-L_t^{m'}\leq I_{m'} \tag{14b}$$

$$f_{mm'}^{u_t^m}\left\{\frac{r_{u_t^m}I_{m'}}{p_{m'}}+\frac{s_{u_t^m}}{d}\cdot\sigma_{mm'}\right\}\leq T_{\text{thr}} \tag{14c}$$

$$f_{mm'}^{u_t^m}\in\{0,1\}, \tag{14d}$$

where the meaning of each constraint can be found in the previous sub-section.

P1 cannot be solved by traditional approaches. First, even at a determined time slot t, **P1** is still a combinatorial optimization problem. Traditional convexification approaches cannot be used because they require global state information of all servers, which is unavailable in distributed scenarios. Second, considering that **P1** is a dynamic optimization problem that needs fast decision speed, heuristic algorithms can neither be used here, as they take a long time to converge. Furthermore, game theory based methods and Lyapunov optimization based methods are the two types of methods that are often used in solving decision-making problems. In game theory based methods, at each time slot, the best response of each participant is obtained from solving static optimization problems, which involves information parsing, screening, and exchanging [30]. Therefore, these methods take a long time to converge and are not suitable for scenarios with high dynamics. Lyapunov optimization based methods [36] are usually applied to problems with long-term budget constraints. However, since the system state changes at each time slot, the value of the Lyapunov function needs to be updated and the corresponding optimization problem needs to be solved again. As a result, these methods are also not appropriate for the problem under consideration.

Noting that the selection of destination BS can be seen as a sequential decision-making problem, another possible solution is to transform problem **P1** into a Markov Decision Process (MDP). There are mainly two solutions for MDP, i.e. dynamic programming and reinforcement learning. Dynamic programming assumes that the model of the system (e.g. state transition matrix) is known by the decision-maker, which is impossible in the considered scenario. As a consequence, reinforcement learning becomes an ideal tool to solve **P1**.

In this article, we solve the problem using a Multi-Agent Reinforcement Learning (MARL) method. Compared with game theory based methods and Lyapunov optimization based methods, MARL based methods have faster execution speed and can handle dynamic and complex problems better. Compared with single-agent reinforcement learning based methods, MARL based methods can model the relationship among different participants more effectively and are more suitable for distributed scenarios.

The procedures of the solution are as follows. First, considering that a BS in an edge network may not frequently share its resource information with others [10], we regard MEC servers in the edge network as a fully-cooperative multi-agent system. Next, the decision-making of the multi-agent system is modeled as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP). Finally, we propose a cooperative task scheduling scheme based on Multi-agent Proximal Policy Optimization (MAPPO) to solve the problem.

IV. Distributed Cooperation Based on MAPPO

This section first introduces the fully-cooperative multi-agent perspective of the considered system, then gives the detail on MAPPO based solution. In the rest of this article, the term cooperative is interchangeable with the term collaborative.

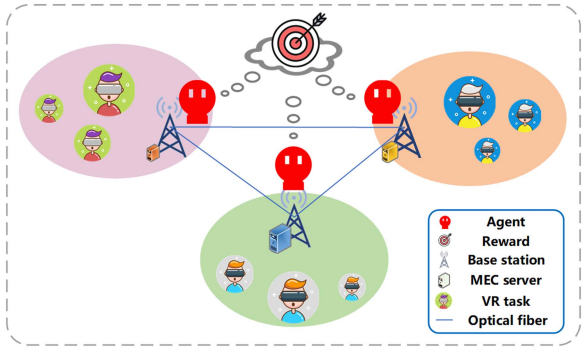

Fig. 3. Edge network as a fully-cooperative multi-agent system.

A. Edge Network as a Multi-Agent System

Multi-agent Systems (MAS) have four typical settings, namely fully-cooperative, fully-competitive, mixed cooperative & competitive, and self-interested. In fully-cooperative settings, agents work together and coordinate their actions, aiming to maximize a shared team reward. In fully-competitive settings, an agent maximizes an individual reward, which is opposite to the reward of its opponent. In mixed settings, agents within a team are cooperative, but different teams are competitive (e.g. robot football game). In self-interested settings, an agent plays a general-sum game with others, where its reward is individual but not opposite to that of others.

In this article, we regard the edge network as a fully-cooperative MAS. As is shown in Fig. 3, each BS (or MEC server) in the edge network can be seen as an agent. Each agent makes offloading decisions for VR tasks independently based on local observations. When making decisions, all agents in the system have a common objective, as is represented in (14). More details about the common objective are provided in the reward design.

Decentralized Partially Observable Markov Decision Process (Dec-POMDP) can be used to describe such fully-cooperative MASs [38]. In Dec-POMDP, a system can be described by $\langle \mathbb{N}, \mathbb{S}, \mathbb{A}, \mathbb{T}, \mathbb{R}, \mathbb{O}, \mathbb{Z}, \gamma \rangle$, where $\mathbb{N}$ is a set of agents, $\mathbb{S}$ is state space, $\mathbb{A}$ is joint action space, $\mathbb{T}$ is state transition function, $\mathbb{R}$ is reward function, $\mathbb{O}$ is the set of joint observation, $\mathbb{Z}$ is observation function, and γ is the discount factor. The objective is to find policies for agents to jointly maximize the expected cumulative reward in a cooperative manner. To do this, the key idea is that each agent has to consider the situation of all agents. Using the multi-agent policy gradient theorem [39], it can be represented as

$$\nabla_{\theta^n} J^n(\theta) = \mathbb{E}_s \left[\mathbb{E}_{a \sim \pi_\theta(\cdot|s)} \left[\nabla_{\theta^n} \log \pi_{\theta^n} (a^n \,|\, s) \cdot Q^{n,\pi_\theta}(s,a) \right] \right], \tag{15}$$

where θ is agent n's policy parameter (or policy network), $\theta = (\theta^n)_{n \in \{1,\ldots,N\}}$ is the collection of policy parameters for all agents, N is the number of agents, and $\pi_\theta = \Pi_{n \in \{1,\ldots,N\}} \pi^n_{\theta^n}(a^n|s)$ is the joint policy of all agents. In (15), the expectation over the joint policy π_θ implies that other agents' policies must be considered by agent n. After getting the policy gradient $\nabla_{\theta^n} J^n(\theta)$, the policy parameter of agent n can be

updated by

$$\theta^n \longleftarrow \theta^n + \alpha \cdot \nabla_{\theta^n} J^n(\theta), \tag{16}$$

where α is the learning rate. Next, we give the necessary description of the Dec-POMDP.

1) State: At time step (slot) t, given the number of agents N, the state space of the system has 3 N elements. Specifically, the state space can be represented as

$$S(t) = \begin{bmatrix} C_t^1 & C_t^2 & \cdots & C_t^N \\ (s_{u_t^1}, r_{u_t^1}) & (s_{u_t^2}, r_{u_t^2}) & \cdots & (s_{u_t^N}, r_{u_t^N}) \end{bmatrix}, \tag{17}$$

where C_t^n is the number of occupied VMs of agent n, and $(s_{u_t^n}, r_{u_t^n})$ is an attribute tuple of the task currently processed by agent n $(1 \le n \le N)$. It is worth noting that at most one task arrives at agent n in time slot t, assuming that the time slot is segmented short enough. When there is no task arrives, the tuple of attributes would be void.

2) Observation: Under the model of Dec-POMDP, the environment is partially observable for each agent. Therefore, the observation of each agent is only a part of the whole state. Specifically, the observation space of all agents can be represented as

$$O(t) = \begin{bmatrix} C_t^1 & (s_{u_t^1}, r_{u_t^1}) & (s_{u_t^2}, r_{u_t^2}) & \cdots & (s_{u_t^N}, r_{u_t^N}) \\ C_t^2 & (s_{u_t^1}, r_{u_t^1}) & (s_{u_t^2}, r_{u_t^2}) & \cdots & (s_{u_t^N}, r_{u_t^N}) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ C_t^N & (s_{u_t^1}, r_{u_t^1}) & (s_{u_t^2}, r_{u_t^2}) & \cdots & (s_{u_t^N}, r_{u_t^N}) \end{bmatrix}, \tag{18}$$

where each row of the matrix is a partial observation of an agent. For agent n, its observations include two parts. The first one is the information of occupied VMs C_t^n, which is a local state of agent n. The second one is attribute tuples of all tasks, which can be sent to all nodes in the edge network through broadcasting.

3) Action: The joint action space of the system can be represented as

$$A(t) = \begin{bmatrix} f_{11}^{u_t^1} & f_{12}^{u_t^1} & \cdots & f_{1N}^{u_t^1} \\ f_{21}^{u_t^2} & f_{22}^{u_t^2} & \cdots & f_{2N}^{u_t^2} \\ \vdots & \vdots & \ddots & \vdots \\ f_{N1}^{u_t^N} & f_{N2}^{u_t^N} & \cdots & f_{NN}^{u_t^N} \end{bmatrix}, \tag{19}$$

where nth row of the matrix is the action space of agent n, noted as $A_n(t)$. For agent n, in time step t, its action is to choose a destination node for the VR task. For example, when agent 1 offloads task u_t^1 from BS 1 to BS 2, $f_{12}^{u_t^1} = 1$. Otherwise $f_{12}^{u_t^1} = 0$.

Note that when all elements of a row in (19) are zero, the corresponding agent will not give any offloading decision. It is because tasks do not arrive at every time step. When there is no task arrived at time step t, zero elements would be a reasonable choice. However, what if an agent still gives a destination node (non-zero element) in this situation? Obviously, an offloading decision is meaningless when there is no task to offload, and it will not bring any meaningful impact to the system directly. However, an invalid action brings interference to the decision-making of other agents, which may influence the learning process adversely. The work of [40] gives a detailed investigation of this issue, where invalid action masking is recommended when the space of invalid action is very large.

4) State Transition Probability: The state transition probability reflects the dynamics of the system model. In this article, the state transition probability cannot be obtained by agents. Specifically, we have the following remark.

Remark 1: In the considered MEC-assisted VR scenario, the state transition probability is unknown by each agent.

Proof: Let $S_n(t)$ be the local state representation of agent n. For any $t \in \mathcal{T}$, the state transition probability of the system can be given as

$$P\left(S(t+1)|S(t), A(t)\right) = \prod_{1 \le n \le N} P\left(S_n(t+1)|S_n(t), A(t)\right). \tag{20}$$

In the RHS of (20), $P(S_n(t+1)|S_n(t), A(t))$ is given by

$$\begin{aligned} &P\left(S_n(t+1)|S_n(t), A(t)\right) \\ &= P\left(C_{t+1}^n|C_t^n, A(t)\right) \times P\left(s_{u_{t+1}^n}|s_{u_t^n}\right) \times P\left(r_{u_{t+1}^n}|r_{u_t^n}\right). \end{aligned} \tag{21}$$

In the RHS of (21), the first term $P(C_{t+1}^n|C_t^n, A(t))$ can be calculated by (9). However, uncertainty exists in the second term $P(s_{u_{t+1}^n}|s_{u_t^n})$ and the third term $P(r_{u_{t+1}^n}|r_{u_t^n})$, as these task attributes follow corresponding Gaussian distributions. Therefore, the system's state transition probability is unknown by each agent. The proof has been completed. ■

In this article, we deal with the unknowable system dynamics by the model-free reinforcement learning paradigm.

5) Reward: The design of the reward function is a crucial and difficult step when solving Dec-POMDP. In this article, the design of the reward function is based on the optimization objective. Considering that the state space and action space are discrete, the reward function is also designed to be discrete. First, we define two auxiliary variables. Let

$$\eta = \max\left(1, \frac{\sum_{m=1}^{M} \sum_{u_t^m=1}^{U_t^m} f_{mm'}^{u_t^m} + C_t^{m'} - L_t^{m'}}{I_{m'}}\right), \tag{22}$$

and

$$\zeta = \max\left(1, \frac{f_{mm'}^{u_t^m} \left\{ \frac{r_{u_t^m} I_{m'}}{p_{m'}} + \frac{s_{u_t^m}}{d} \cdot \sigma_{mm'} \right\}}{T_{\text{thr}}}\right), \tag{23}$$

where $\max(\cdot)$ means to choose a larger one from the two terms. Next, we define the reward function as

$$\begin{aligned} R(t) = R_1 \cdot \delta\left(\eta \cdot \zeta - 1\right) + R_2 \left[1 - \delta\left(\sum_{m'=1}^{M} f_{mm'}^{u_t^m} - 1\right)\right] \\ + R_3 \cdot \left[1 - \delta\left(\eta - 1\right)\right], \end{aligned} \tag{24}$$

where $\delta(\cdot)$ is the Dirac function, R_1, R_2, and R_3 are coefficient hyperparameters. After making an offloading decision, an agent

will get an instant reward. Positive rewards can be seen as incentives for agents to take appropriate actions, while negative rewards can be seen as penalties that agents receive when they take reckless actions. In (24), the first term $R_1 \cdot \delta(\eta \cdot \zeta - 1)$ is a positive reward. An agent gets this reward when its action satisfies (14b) and (14c). The second term $R_2[1 - \delta(\sum_{m'=1}^{M} f_{mm'}^{u_t^m} - 1)]$ is a penalty that an agent gets when its action does not satisfy (14a). The third term $R_3 \cdot [1 - \delta(\eta - 1)]$ is a penalty that an agent gets when its action does not satisfy (14b). The third term of the reward function makes agents cautious to adopt a greedy strategy that keeps scheduling tasks to a certain server. This is critical to ensure the load balance of the edge network.

To achieve cooperation, different agents share an identical reward, i.e. $r_t^1 = r_t^2 = \cdots = r_t^N = R(t)$. The common reward distinguishes the cooperative MAS from the self-interested MAS. For example, when sharing a common reward, an agent cannot always offload tasks to a fixed destination node, because if the VM of this destination node overflows, the rewards of all agents will drop. Instead, an agent should consider what actions other agents will take (based on common observations), and then take corresponding cooperative actions.

B. MAPPO Based Solution

In this article, we propose a cooperative task scheduling scheme based on MAPPO [41] to solve the aforementioned Dec-POMDP. MAPPO is an extended form of Proximal Policy Optimization (PPO). Next, we first introduce the basic idea of PPO, then present the MAPPO based solution to the problem.

1) PPO: PPO is a deep reinforcement learning algorithm based on policy gradient, facing both continuous and discrete action space. It adopts the Actor-Critic framework, where the Actor generates actions and the Critic assesses the actions. Compared with off-policy algorithms that support experience replay, an important shortcoming of the vanilla Actor-Critic framework is its low sample effectiveness, which means that it needs huge interaction with environments before converging. To deal with this problem, PPO adopts importance sampling, which allows a tiny difference between the target policy and the behavior policy. To be more specific, it constrains the difference between the old and new policies by using a clip function. When the change of policy exceeds a certain threshold, the clip function would avoid it from further growing. There are two networks in PPO, namely the policy network and the value network. The objective of the value network is to provide the agent's action with an assessment, which should be close to the actual reward given by the environment. This objective can be represented as

$$\min_{\omega} \left(R(t) + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)\right)^2, \tag{25}$$

where ω is parameters of the value network and $V_\omega(s_t)$ is the state value function. Let

$$\delta_t = R(t) + \gamma V_\omega(s_{t+1}) - V_\omega(s_t) \tag{26}$$

be the TD-error, then we define the advantage function using Generalized Advantage Estimation (GAE) [42]:

$$A_t = \sum_{t'=0}^{\infty} (\gamma\psi)^{t'} \delta_{t+t'}, \tag{27}$$

where γ is the discounted factor and ψ is used to adjust the bias-variance trade-off. Next, we can represent the objective of the policy network as

$$\max_{\theta} \mathbb{E}_t \left[\min\left(r_t(\theta)A_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)A_t\right)\right], \tag{28}$$

where θ is parameters of the policy network and $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. The clip function constrains the value of $r_t(\theta)$ within the range of $[1-\epsilon, 1+\epsilon]$, where ϵ is a hyperparameter. During the training, the policy network and the value network are updated alternately.

2) Mappo: MAPPO is a Centralized Training & Decentralized Execution (CTDE) algorithm that extends PPO to multi-agent scenarios. CTDE framework becomes favorable in recent years because of its convergence stability and fast execution speed [43], [44], [45], [46], [47]. In the training stage, communication between agents is needed for sharing local observations. Afterwards, the learned policy is kept by each agent and executed in a fully-distributed fashion.

In MAPPO, each agent learns a policy π_θ to map agent observations to the distribution of actions. Each agent also has a value function $V_\omega(s)$, which is only utilized during the training for collecting extra global information. In practice, π_θ and $V_\omega(s)$ can be parameterized by two neural networks.

The policy network (referred to as an actor) is trained to maximize a loss function $L(\theta)$, which can be written as

$$L(\theta) = \mathbb{E}_i\left[\mathbb{E}_n\left[\min\left(r_{\theta,i}^n A_i^n, \underbrace{\text{clip}(r_{\theta,i}^n, 1-\epsilon, 1+\epsilon)A_i^n}_{\text{clipped value function with an advantage}}\right) + \underbrace{\sigma S\left[\pi_\theta(o_i^n)\right]}_{\text{entropy regularization}}\right]\right], \tag{29}$$

where i denotes a batch of data, n denotes an agent, $r_{\theta,i}^n = \frac{\pi_\theta(a_i^n|o_i^n)}{\pi_{\theta_{\text{old}}}(a_i^n|o_i^n)}$, and A_i^n is the advantage function that can be estimated by GAE. $\sigma S[\pi_\theta(o_i^n)]$ is an entropy regularization term that encourages agents to explore better policies, where $S[\pi_\theta(o^n)] = -\sum_{a^n} \pi_\theta(a^n|o^n) \cdot \log \pi_\theta(a^n|o^n)$ represents the policy entropy and σ is a coefficient parameter. $\min(\cdot)$ means to choose a smaller one from the two terms. The value network (referred to as a critic) is trained to minimize a loss function $L(\omega)$, which can be represented as

$$L(\omega) = \mathbb{E}_i\left[\mathbb{E}_n\left[\max\left[\left(V_\omega(s_i^n) - \hat{R}_i\right)^2, \left(\text{clip}(V_\omega(s_i^n), V_{\omega_{\text{old}}}(s_i^n)-\epsilon, V_{\omega_{\text{old}}}(s_i^n)+\epsilon) - \hat{R}_i\right)^2\right]\right]\right], \tag{30}$$

where $\max(\cdot)$ means to choose a larger one from the two terms, $\hat{R}_i$ is the reward-to-go [48]. $V_{\omega_{\text{old}}}(s_i^n)$ is the state value output when network parameter is ω_{old}. To limit the update of the value network to a small range, the clip function is also used here.

Fig. 4 illustrates the framework of the proposed cooperative task scheduling based on MAPPO. In the vanilla MAPPO algorithm, Recurrent Neural Networks (RNNs) are used in policy

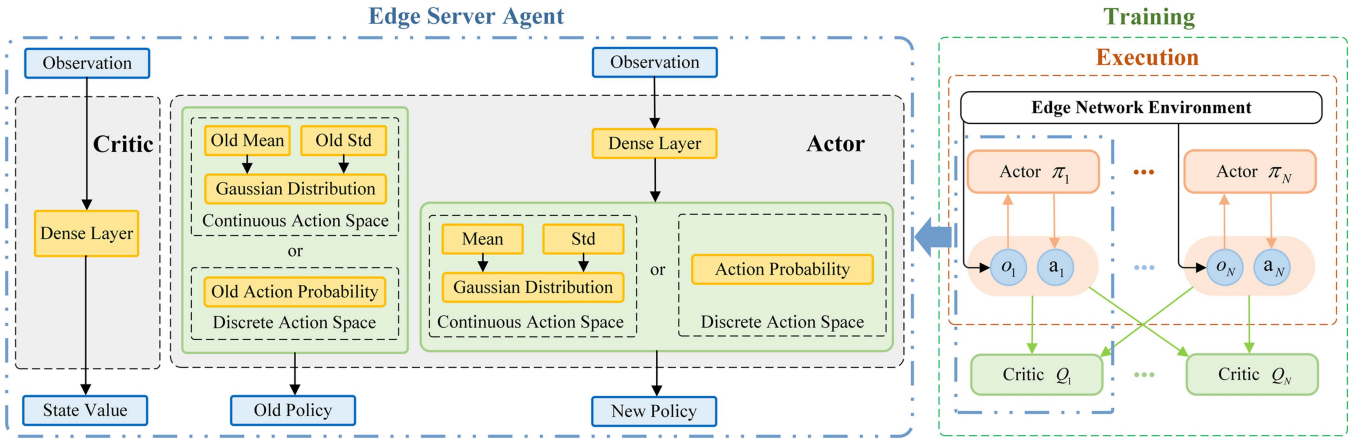

Fig. 4. Framework of MAPPO based cooperative task scheduling.

and value networks. The purpose is to provide agents with better policies by collecting historical observations. In this article's considered scenario, however, users may randomly terminate VR tasks, which leads to a poor correlation between historical observations and current scheduling decisions. Therefore, we use fully-connected dense layers rather than the more complicated RNNs to obtain a better trade-off between convergence speed and reward value.

Denote the number of neurons in the ith layer of the actor as L_i^a and the number of neurons in the jth layer of the critic as L_j^c, then the computational complexity of the actor can be written as $\mathcal{D}_1 = \mathcal{O}(\sum_{i=0}^{I} L_i^a \cdot L_{i+1}^a)$, where I is the number of hidden layers of the actor. Similarly, the computational complexity of the critic can be written as $\mathcal{D}_2 = \mathcal{O}(\sum_{j=1}^{J} L_j^c \cdot L_{j+1}^c)$, where J is the number of hidden layers of the critic. It should be noticed that the 0th layer of the neural network refers to the input layer, and the last layer refers to the output layer. Considering that there are M agents, each of which has an actor and a critic, the total computational complexity of execution can be given as $M \times (\mathcal{D}_1 + \mathcal{D}_2)$.

According to the discussion above, the MAPPO based solution to the Dec-POMDP can be summarized as Algorithm 1. In Algorithm 1, $batch\_size$ refers to the number of samples collected in one iteration, and $mini\_batch$ refers to the number of samples used in one gradient descent optimization.

Algorithm 1: MAPPO based solution to the Dec-POMDP.

Initialization: Set the structure of networks and the learning rate α. Initialize policy network's parameters θ and value network's parameters ω.
for each $iteration$ **do**
  Set data buffer $B = \{\}$;
  for $i = 1, 2, \ldots, batch\_size$ **do**
    Set $\tau = [\,]$, the buffer of interaction trajectory;
    for $t = 1, 2, \ldots, T$ **do**
      Each agent n selects scheduling action $A_n(t)$;
      Agents execute joint scheduling actions $A(t)$;
      Agents get $R(t), S(t+1), O(t+1)$;
      $\tau += [S(t), O(t), A(t), R(t), S(t+1), O(t+1)]$;
    end for
    Estimate advantage A using GAE on τ;
    Compute reward-to-go $\hat{R}$ on τ;
    $B += (\tau, A, \hat{R})$;
  end for
  for $mini\_batch$ $k = 1, 2, \ldots, K$ **do**
    $b \leftarrow$ randomly select $mini\_batch$ from B;
  end for
  Update θ on $L(\theta)$ with data b using Adam [49];
  Update ω on $L(\omega)$ with data b using Adam;
end for
Output: Task scheduling policy π_θ.

V. Simulations and Discussions

A. Simulation Settings

Simulations are conducted to measure the performance of the proposed cooperative scheduling scheme. The following simulations are conducted in a computer with an Intel Core i5 CPU and 8 GB RAM. The coding environment is PyCharm CE, running PyTorch 1.5.1 and Python 3.7. In the simulations, we consider an edge network that consists of 3 BSs, each of which is equipped with a MEC server, and a BS is connected with others via optical fiber. Under each BS, there are VR users with heterogeneous task attributes, and the attributes follow Gaussian Distributions. The arrival and leaving time of each user is randomly selected and we use the intensity factor to control this process.

Table II concludes the main parameter settings and reference. The number of neurons and hidden layers applies for both the policy network and the value network. Noting that positive rewards attract agents so much that they may adopt counter-intuitive greedy policies, we only use a small positive term of R_1. Unless otherwise specified, the maximum tolerable delay T_{thr} is 20 ms.

For comparison, we consider the following scheduling schemes in MEC-assisted VR rendering. The fully-distributed decision-making is ensured in all of the schemes.

TABLE II
PARAMETER SETTINGS

Parameter	Value
Number of agents	3
Learning rate	5×10^{-4}
Number of neurons in each layer	128
Number of hidden layers	2
PPO epoch	15
Number of rollout threads	from 2 to 64
R_1	1
R_2	-5
R_3	-10
β	0.04
λ	from 0.2 to 0.4
$r_{u_t^m}$	from 6 to 24 million [6]
$s_{u_t^m}$	from 12.8 to 51.2 Mb [6]
p_m	from 10 to 40 GHz [9]
I_m	from 10 to 20
d	10Gbps [23]
T_{thr}	from 10 to 35 ms [5]

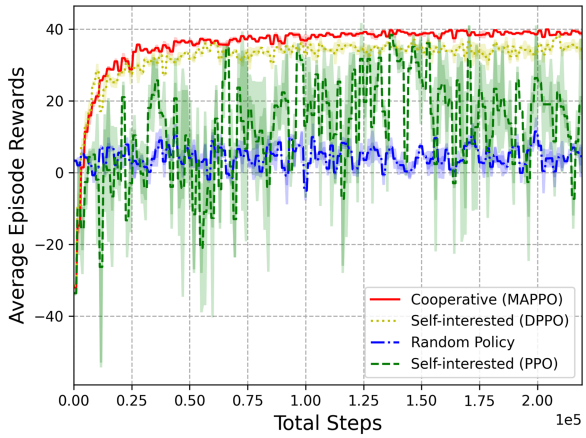

Fig. 5. Learning curve of different scheduling schemes, i.e. MAPPO based cooperative scheme, DPPO based self-interested scheme, random policy, and PPO based self-interested scheme.

- Cooperative scheduling scheme based on MAPPO, as is proposed in this article. In this scheme, different BS collaboratively schedule VR rendering tasks to maximize the QoE of all users in the network.
- Self-interested scheduling scheme based on DPPO (and PPO), which is one of the state-of-the-art single-agent reinforcement learning algorithms. In this scheme, each BS selects peer nodes for rendering to maximize its own users' QoE.
- Random policy, where each BS randomly selects a peer node (including itself) for rendering.

B. Simulation Results

Fig. 5 shows the learning curve of different scheduling schemes for MEC-assisted VR rendering: 1) the cooperative scheme based on MAPPO, 2) the self-interested scheme based on DPPO & PPO, and 3) the random policy. To eliminate the influence of random numbers, we use three random seeds for each scheme in the simulation. The shadow of each curve represents the variance among different seeds, where the obvious shadow indicates a large variance.

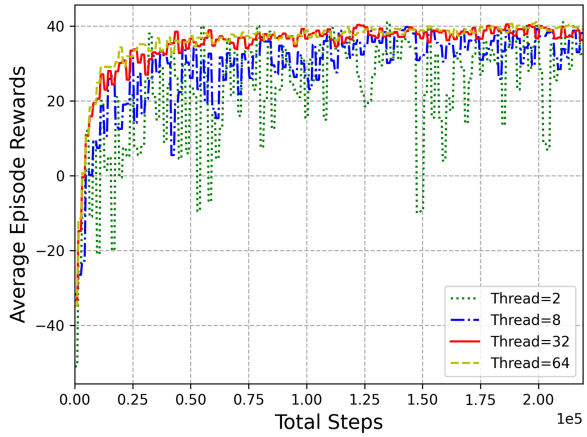

Fig. 6. Learning curve for different number of threads in MAPPO.

It can be seen that MAPPO based cooperative scheme converges within 70 thousand steps and it has the highest average rewards. Besides, the cooperative scheme also has an extremely small variance when using different random seeds, indicating that this scheme has outstanding robustness. For the self-interested scheme, we implement both DPPO and PPO algorithms. The learning curve shows that the PPO based scheme does not converge even after 200 thousand steps, while DPPO converges almost simultaneously with the MAPPO. The reason is that DPPO uses multi-threading during the training stage, which brings more chances of exploration for better policies. However, DPPO still has smaller rewards and bigger variance than the MAPPO based scheme. This result reveals the advantage of cooperative schemes over self-interested schemes. The reason lies in that: 1) In a cooperative scheme, agents fully utilize the attribute tuples in their observation and evaluate the interest of other agents. 2) By sharing one common objective, agents collaboratively maximize their rewards and avoid decision conflicts. However, the cooperation among agents does not always bring benefits, of which the detail will be introduced later.

Fig. 6 illustrates the learning curves when using four different threads in MAPPO. In this article, we apply multi-threading for the training of MAPPO. Multi-threading allows many rollout workers to interact with different environments to collect more trajectories, thus increasing the possibility of finding good policies. When using 2 threads to train the agents, the curve does not converge well. The learning curve begins to converge when the number of threads is 8, but the jitter of average episode rewards is still relatively large until using 32 threads. Although using 64 threads brings an even smaller jitter, it also makes the time of training longer. Finally, we use 32 threads for the rest of the simulations, as this value has a good trade-off between the convergence and the cost of time.

Next, we test the performance of well-trained agents by making scheduling decisions for the randomly generated VR tasks, of which the results are shown in Fig. 7. Here we use the following two evaluation indicators.

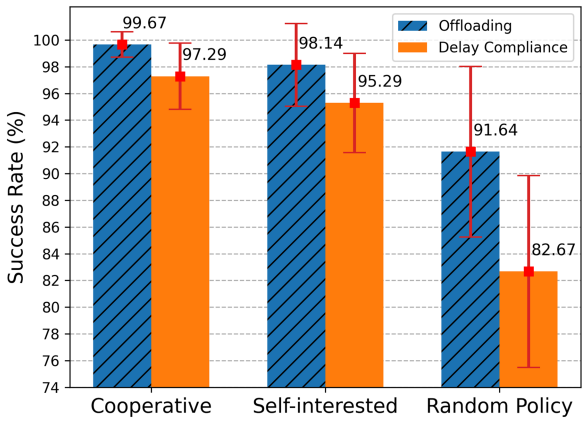

Fig. 7. Offloading success rate and delay compliance rate of the three scheduling schemes.

- The **offloading success rate**, where the success means that: the number of VMs in the destination node cannot exceed the limitation (overflow), as is represented in (14b).
- The **delay compliance rate**, which means that: 1) the number of VMs in the destination node does not overflow, and 2) the delay of rendering a frame is smaller than the maximum tolerated value, as is represented in (14b) and (14c).

We obtain each test result by scheduling 42 randomly generated VR tasks, and then repeat this process 100 times to get an average value and variance. In Fig. 7, the bars on top of the histogram demonstrate the variance. The result shows that the MAPPO based cooperative scheme has the highest success rate and the smallest variance, which is consistent with the result shown in Fig. 5. Note that the PPO based self-interested scheduling scheme is not considered here because it does not converge.

Fig. 8 shows the resource utilization versus episode steps under the three scheduling schemes, reflecting the load balance of the edge network. As the episode steps grow, random tasks arrive at each BS. In the simulations, BS1 has a total CPU frequency of 10 GHz and a VM limit of 10; BS 2 has a total CPU frequency of 20 GHz and a VM limit of 15; BS 3 has a total CPU frequency of 40 GHz and a VM limit of 20. Resource utilization reflects the percentage of VMs that are used to the total number of VMs. A rise in resource utilization represents that a VM is allocated for a task, while a decline represents that a task is ended and its VM is released. When the resource utilization neither rises nor declines, there are two possible events: 1) There is no task allocated to the corresponding BS. 2) The arrival and the leaving of a task happen at the same time.

It can be observed from Fig. 8(a) that the three curves grow steadily and never reach 100%, which indicates a good balance of workload. This is because the agents have learned a cooperative policy to properly arrange destination nodes so as to avoid the overflow of a single MEC server. In contrast, in Fig. 8(b), the self-interested agents fall into a trap caused by greed, i.e. the mismatch between the task's requirement and the resource of VM. In this situation, the VM of the powerful server (BS 3) is easy to overflow, but the utilization rate of servers with poor resources keeps low (BS 1 and BS 2). In Fig. 8(c), the destination node is randomly selected by the agents, which leads to the overflow of the BS with the smallest VM limit (BS 1).

Figs. 9 and 10 show the offloading success rate and the delay compliance rate versus the task arrival rate λ. When plotting these two figures, we train the agents using different task arrival rates and then test them by randomly generated tasks. From these figures, it can be seen that as λ increases, the three lines both decline. This is because as λ grows, the number of arrived tasks per second becomes bigger, and it makes the VMs on the MEC server easier to overflow, leading to the drop of offloading success rate and delay compliance rate. Besides, the cooperative scheme based on MAPPO and the self-interested scheme based on DPPO outperform random policy for all tasks arrival rate settings.

We also notice an interesting conclusion from Figs. 9 and 10. The cooperative scheme outperforms the self-interested scheme for most λ settings, but not always. Specifically, the cooperative scheme has a better performance than the self-interested scheme when λ is between 0.2 and 0.34. The biggest gap between the cooperative scheme and the self-interested scheme appears when $\lambda = 0.32$. But when λ is greater than 0.34, the self-interested scheme has a small advantage over the cooperative scheme. A possible reason for this is that in the cooperation scheme, an agent has to evaluate other agents' policies before taking an action, as is described in (15), which introduce an extra interference for the offloading decision, especially when the task arrival rate is big and MEC servers are facing huge pressure. In this situation, a self-interested policy is better than cooperating with others. However, the proposed cooperative scheme is still useful in most scenarios, because $\lambda \geq 0.34$ is an extreme setting in the simulation, where the offloading success rate and the delay compliance rate of all schemes are lower than 90%. In practical scenarios, this extreme situation should be avoided by expanding the scale of MEC servers.

Fig. 11 is the offloading success rate versus the maximum tolerated delay for the three scheduling schemes. The MAPPO based cooperative scheduling scheme has the highest offloading success rate for all maximum tolerated delay settings, and it has only a tiny fluctuation on the curve. The curve of random policy has no fluctuation for all x-axis values. The reason is that in order to control the variables we use a fixed random seed, which makes the random policy generates the same offloading decisions in each round of testing.

Fig. 12 is the delay compliance rate versus the maximum tolerated delay. The figure shows that as the maximum tolerated delay increases, the delay compliance rate of the three schemes all grows. This is because a greater maximum tolerated delay reduces the requirements for MEC servers' hardware and gives greater tolerance to the scheduling strategy. In the three scheduling schemes, the cooperative scheme has a better overall performance than others. It is worth noting that the curve of DPPO has a drop between 25 and 30 ms. We hold that the reason lies in the overfitting of deep neural networks. This situation did not occur in the MAPPO based cooperative scheme, which can be seen as a demonstration of its good robustness.

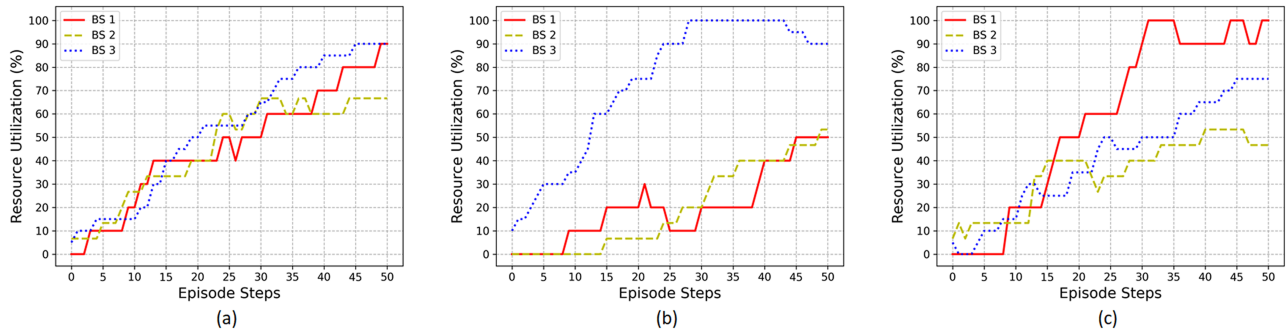

Fig. 8. Resource utilization under the three scheduling schemes. It reflects the load balance of the edge network. (a) Cooperative task scheduling. (b) Self-interested task scheduling. (c) Random policy.

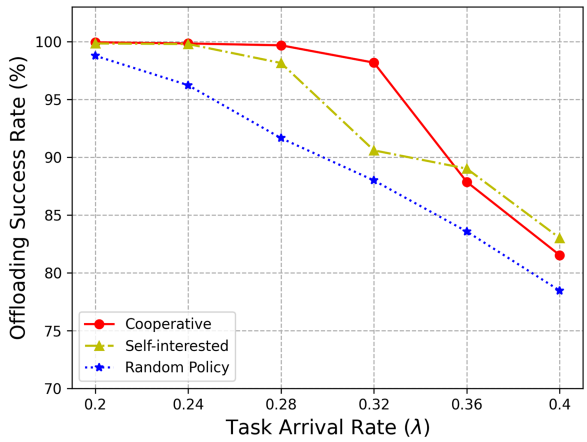

Fig. 9. Offloading success rate of different scheduling schemes.

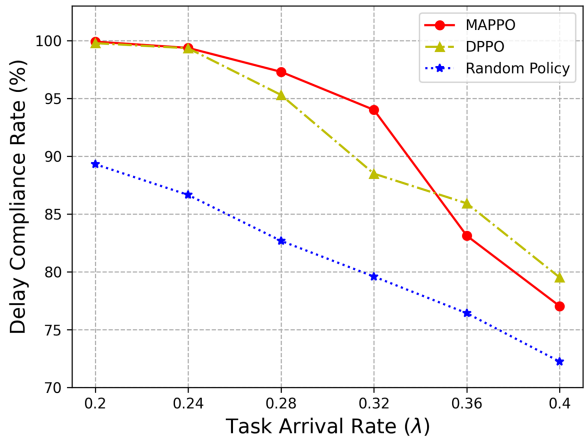

Fig. 10. Delay compliance rate of different scheduling schemes.

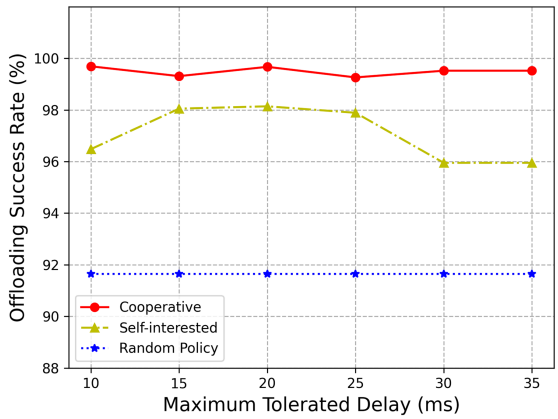

Fig. 11. Offloading success rate versus the maximum tolerated delay for the three scheduling schemes.

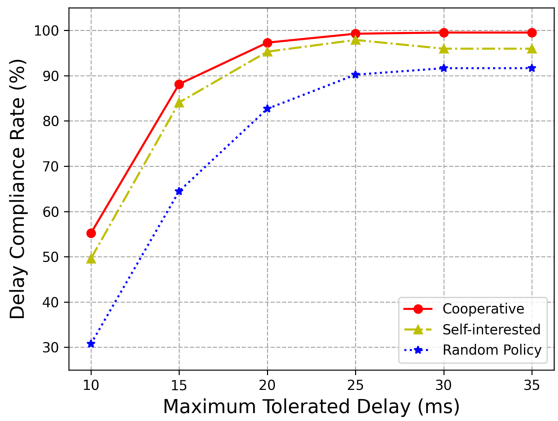

Fig. 12. Delay compliance rate versus the maximum tolerated delay for the three scheduling schemes.

C. Discussions on Scalability

When deployed in the real world, existing MARL algorithms may suffer from scalability issues. On the one hand, the convergence of the algorithm becomes worse when the number of agents increases, since the size of the global state/action space may be exponential in the number of agents during the training phase. To this end, the existing study proposes a knowledge transfer method between agents [50], which transfers the knowledge learned by some agents to other agents, thereby speeding up the convergence. Other studies have proposed MARL methods based on curriculum learning [51]. The idea is to increase the learning difficulty step by step with the expansion of the agent scale, thereby reducing the training time.

On the other hand, the dynamic change of agent scale also brings challenges to the application of deep reinforcement learning algorithms. Generally, deep reinforcement learning utilizes neural networks to approximate optimal policies, and neural networks require fixed-dimensional state representations

as input. However, due to the uncertainty of the agent scale, the dimension of the system state may change dynamically. This contradiction limits the scalability of deep reinforcement learning algorithms, making it difficult for the learned policy to adapt to the dynamic changes in agent scale. In order to deal with the problem of variable-length input, existing studies have employed graphic representation [52], Seq2Seq models [53], average embedding [54], attention mechanisms [55], graph neural networks [56], and other techniques to encode variable-length input into fixed-dimensional feature vectors.

In the scenario considered in this article, the number of edge servers may be large, and network operators may adjust the edge network scale by adding or removing servers. Therefore, when deploying the proposed multi-agent based task scheduling scheme in practice, other feature extraction and learning techniques should be considered together to ensure algorithm convergence and scalability.

VI. Conclusion

In this article, we proposed a cooperative task scheduling scheme for MEC-assisted VR, where all MEC servers cooperatively choose offloading actions to maximize the number of delay-satisfied tasks of the entire network. To this end, we first formulated an optimization problem, considering realistic features such as distributed scenarios without a central unit, random arrival and leaving of each task, as well as parallel multitasking. Next, we introduced the fully-cooperative multi-agent perspective of the system and proposed a MAPPO based cooperative scheduling scheme. Finally, we conducted detailed simulations to compare the proposed cooperative scheduling scheme with a self-interested scheduling scheme and a random policy. The results demonstrated the superiority of the proposed scheme in terms of offloading success rate, delay compliance rate, and load balance. In our follow-up research, we will further consider the mobility pattern of VR users and the potential handovers between different base stations. Additionally, we aim to enhance the scalability of MARL based algorithms by incorporating other feature extraction and training techniques.

References

[1] Z. Lai, Y. C. Hu, Y. Cui, L. Sun, N. Dai, and H. Lee, "Furion: Engineering high-quality immersive virtual reality on today's mobile devices," *IEEE Trans. Mobile Comput.*, vol. 19, no. 7, pp. 1586–1602, Jul. 2020, doi: 10.1109/TMC.2019.2913364.

[2] Samsung, "The next hyper-connected experience for all," Jul. 2020. Accessed: Jun. 14, 2023. [Online] Available: https://news.samsung.com/uk/samsungs-6g-white-paper-lays-out-the-companys-vision-for-the-next-generation-of-communications-technology

[3] "M&M, virtual reality market," Aug. 2020. Accessed: Jun. 14, 2023. [Online] Available: https://www.marketsandmarkets.com/Market-Reports/reality-applications-market-458.html

[4] Y. Liu, J. Liu, A. Argyriou, L. Wang, and Z. Xu, "Rendering-aware VR video caching over multi-cell MEC networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 3, pp. 2728–2742, Mar. 2021, doi: 10.1109/TVT.2021.3057684.

[5] C. Zheng, S. Liu, Y. Huang, and L. Yang, "Hybrid policy learning for energy-latency tradeoff in MEC-Assisted VR video service," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9006–9021, Sep. 2021, doi: 10.1109/TVT.2021.3099129.

[6] Y. Liu, J. Liu, A. Argyriou, and S. Ci, "MEC-Assisted panoramic VR video streaming over millimeter wave mobile networks," *IEEE Trans. Multimedia*, vol. 21, pp. 1302–1316, 2019, doi: 10.1109/TMM.2018.2876044.

[7] F. Guo, F. R. Yu, H. Zhang, H. Ji, V. C. M. Leung, and X. Li, "An adaptive wireless virtual reality framework in future wireless networks: A. distributed learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8514–8528, Aug. 2020, doi: 10.1109/TVT.2020.2995877.

[8] J. Du, F. R. Yu, G. Lu, J. Wang, J. Jiang, and X. Chu, "MEC-Assisted immersive VR video streaming over terahertz wireless networks: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9517–9529, Oct. 2020, doi: 10.1109/JIOT.2020.3003449.

[9] X. Liu and Y. Deng, "Learning-based prediction, rendering and association optimization for MEC-Enabled wireless virtual reality (VR) networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6356–6370, Oct. 2021, doi: 10.1109/TWC.2021.3073623.

[10] M. Chen, O. Semiari, W. Saad, X. Liu, and C. Yin, "Federated echo state learning for minimizing breaks in presence in wireless virtual reality networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 177–191, Jan. 2020, doi: 10.1109/TWC.2019.2942929.

[11] Huawei-ilab,"Whitepaper on cloud VR solution," 2018. Accessed: Jun. 14, 2023. [Online]. Available: http://www.huawei.com/

[12] X. Wei, C. Yang, and S. Han, "Prediction, communication, and computing duration optimization for VR video streaming," *IEEE Trans. Commun.*, vol. 69, no. 3, pp. 1947–1959, Mar. 2021, doi: 10.1109/TCOMM.2020.3040282.

[13] P. Lin, Q. Song, F. R. Yu, D. Wang, and L. Guo, "Task offloading for wireless VR-Enabled medical treatment with blockchain security using collective reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 21, pp. 15749–15761, Nov. 2021, doi: 10.1109/JIOT.2021.3051419.

[14] X. Wang, X. Ren, C. Qiu, Y. Cao, T. Taleb, and V. C. M. Leung, "Net-in-AI: A. computing-power networking framework with adaptability, flexibility, and profitability for ubiquitous AI," *IEEE Netw.*, vol. 35, no. 1, pp. 280–288, Jan./Feb. 2021, doi: 10.1109/MNET.011.2000319.

[15] L. Chen, S. Zhou, and J. Xu, "Computation peer offloading for energy-constrained mobile edge computing in small-cell networks," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1619–1632, Aug. 2018, doi: 10.1109/TNET.2018.2841758.

[16] Y. Li, X. Wang, X. Gan, H. Jin, L. Fu, and X. Wang, "Learning-aided computation offloading for trusted collaborative mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 12, pp. 2833–2849, Dec. 2020, doi: 10.1109/TMC.2019.2934103.

[17] X. Wang, Z. Ning, and S. Guo, "Multi-agent imitation learning for pervasive edge computing: A. decentralized computation offloading algorithm," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 411–425, Feb. 2021, doi: 10.1109/TPDS.2020.3023936.

[18] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop multi-task partial computation offloading in collaborative edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1133–1145, May 2021, doi: 10.1109/TPDS.2020.3042224.

[19] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multihop offloading of multiple DAG tasks in collaborative edge computing," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4893–4905, Mar. 2021, doi: 10.1109/JIOT.2020.3030926.

[20] M. Li, J. Gao, L. Zhao, and X. Shen, "Deep reinforcement learning for collaborative edge computing in vehicular networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1122–1135, Dec. 2020, doi: 10.1109/TCCN.2020.3003036.

[21] A. Ndikumana et al., "Joint communication, computation, caching, and control in Big Data multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 6, pp. 1359–1374, Jun. 2020, doi: 10.1109/TMC.2019.2908403.

[22] X. He and S. Wang, "Peer offloading in mobile-edge computing with worst case response time guarantees," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2722–2735, Feb. 2021, doi: 10.1109/JIOT.2020.3019492.

[23] X. Wang, Z. Ning, L. Guo, S. Guo, X. Gao, and G. Wang, "Online learning for distributed computation offloading in wireless powered mobile edge computing networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1841–1855, Aug. 2022, doi: 10.1109/TPDS.2021.3129618.

[24] Q. Peng, C. Wu, Y. Xia, Y. Ma, X. Wang, and N. Jiang, "DoSRA: A decentralized approach to online edge task scheduling and resource allocation," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4677–4692, Mar. 2022, doi: 10.1109/JIOT.2021.3107431.

[25] W. Zhang, G. Zhang, and S. Mao, "Joint parallel offloading and load balancing for Cooperative-MEC systems with delay constraints," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4249–4263, Apr. 2022, doi: 10.1109/TVT.2022.3143425.

[26] Z. Jing, Q. Yang, Y. Wu, M. Qin, K. Sup Kwak, and X. Wang, "Adaptive cooperative task offloading for energy-efficient small cell MEC networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 292–297, doi: 10.1109/WCNC51071.2022.9771874.
[27] X. He, S. Wang, and X. Wang, "Providing worst-case latency guarantees with collaborative edge servers," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2955–2971, May 2023, doi: 10.1109/TMC.2021.3133306.
[28] J. Ren et al., "An efficient two-layer task offloading scheme for MEC system with multiple services providers," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1519–1528, doi: 10.1109/INFOCOM48880.2022.9796843.
[29] L. Yuan et al., "CoopEdge : Enabling decentralized, secure and cooperative multi-access edge computing based on blockchain," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 894–908, Mar. 2023, doi: 10.1109/TPDS.2022.3231296.
[30] P. Lang, D. Tian, X. Duan, J. Zhou, Z. Sheng, and V. C. M. Leung, "Cooperative computation offloading in blockchain-based vehicular edge computing networks," *IEEE Trans. Intell. Veh.*, vol. 7, no. 3, pp. 783–798, Sep. 2022, doi: 10.1109/TIV.2022.3190308.
[31] S. Ohl, M. Willert, and O. Staadt, "Latency in distributed acquisition and rendering for telepresence systems," *IEEE Trans. Vis. Comput. Graph.*, vol. 21, no. 12, pp. 1442–1448, Dec. 2015, doi: 10.1109/TVCG.2015.2407403.
[32] B. G. Chun et al., "Clonecloud: Elastic execution between mobile device and cloud," in *Proc. 6th Conf. Comput. Syst.*. 2011, pp. 301–314.
[33] Y. Han, D. Guo, W. Cai, X. Wang, and V. Leung, "Virtual machine placement optimization in mobile cloud gaming through QoE-Oriented resource competition," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 2204–2218, Jul.–Sep.– 2022, doi: 10.1109/TCC.2020.3002023.
[34] K.-T. Seo, H.-S. Hwang, I.-Y. Moon, O.-Y. Kwon, and B.-J. Kim, "Performance comparison analysis of linux container and virtual machine for building cloud," *Adv. Sci. Technol. Lett.*, vol. 66, pp. 105–111, 2014.
[35] Q. Zhang, L. Liu, C. Pu, Q. Dou, L. Wu, and W. Zhou, "A comparative study of containers and virtual machines in Big Data environment," in *Proc. IEEE 11th Int. Conf. Cloud Comput.*, 2018, pp. 178–185, doi: 10.1109/CLOUD.2018.00030.
[36] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2333–2345, Oct. 2018, doi: 10.1109/JSAC.2018.2869954.
[37] H. Guo and J. Liu, "Collaborative computation offloading for multiaccess edge computing over fiber–wireless networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 5, pp. 4514–4526, May 2018, doi: 10.1109/TVT.2018.2790421.
[38] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Cham, Switzerland: Springer, 2016.
[39] Y. Yang and J. Wang, "An overview of multi-agent reinforcement learning from game theoretical perspective," 2020, *arXiv:2011.00583*.
[40] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. Int. FLAIRS Conf.*, May 2022, vol. 35. [Online]. Available: https://doi.org/10.32473/flairs.v35i.130584
[41] Y. Chao et al., "The surprising effectiveness of PPO in cooperative, multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 24611–24624.
[42] J. Schulman et al., "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
[43] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-Assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021, doi: 10.1109/JSAC.2020.3036962.
[44] Y. Gong, H. Yao, J. Wang, L. Jiang, and F. R. Yu, "Multi-agent driven resource allocation and interference management for deep edge networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 2, pp. 2018-2030, Feb. 2022, doi: 10.1109/TVT.2021.3134467.
[45] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30.
[46] T. Rashid et al., "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, pp. 7234–7284, 2020.
[47] S. Omidshafiei et al., "Deep decentralized multi-task multi-agent reinforcement learning under partial observability," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2681–2690.
[48] OpenAI, Spinning Up. Accessed: Jun. 14, 2023. [Online]. Available: https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html
[49] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Comput. Sci.*, 2014.
[50] S. Wadhwania, D. -K. Kim, S. Omidshafiei, and J. P. How, "Policy distillation and value matching in multiagent reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 8193–8200, doi: 10.1109/IROS40897.2019.8967849.
[51] K. Shao, Y. Zhu, and D. Zhao, "StarCraft micromanagement with reinforcement learning and curriculum transfer learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 3, no. 1, pp. 73–84, Feb. 2019, doi: 10.1109/TETCI.2018.2823329.
[52] L. Han et al., "Grid-wise control for multi-agent reinforcement learning in video game ai," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2576–2585.
[53] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 3052–3059, doi: 10.1109/IROS.2018.8593871.
[54] Y. Yang et al., "Mean field multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5571–5580.
[55] S. Iqbal et al., "Randomized entity-wise factorization for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4596–4606.
[56] Z. Ye, K. Wang, Y. Chen, X. Jiang, and G. Song, "Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4056–4069, Jul. 2023, doi: 10.1109/TMC.2022.3146881.

Ziqi Li received the B.S. and M.E. degrees in electronics and communication engineering from Shaanxi Normal University, Xi'an, China, in 2017 and 2019, respectively. He is currently working toward the Ph.D. degree with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing, China. His research interests include multiaccess edge computing, distributed machine learning, and future communication networks.

Heli Zhang received the B.S. degree in communication engineering from Central South University, Changsha, China, in 2009, and the Ph.D. degree in communication and information System from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2014. From 2014 to 2018, she was the Lecturer with the School of Information and Communication Engineering, BUPT. From 2018, she has been the Associate Professor with the School of Information and Communication Engineering, BUPT. Her research interests include heterogeneous networks, long-term evolution/fifth generation and Internet of Things. She was a Reviewer for Journals of IEEE WIRELESS COMMUNICATIONS, IEEE COMMUNICATION MAGAZINE, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE COMMUNICATION LETTERS, and IEEE TRANSACTIONS ON NETWORKING. She participated in many National projects funded by National Science and Technology Major Project, National 863 High-tech and National Natural Science Foundation of China, and cooperated with many Corporations in research.

Xi Li received the B.E. and Ph.D. degrees in the major of communication and information system from the Beijing University of Posts and Telecommunications (BUPT), in 2005 and 2010, respectively. In 2017 and 2018, she was a Visiting Scholar with The University of British Columbia, Vancouver, BC, Canada. She is currently a Professor with the School of Information and Communication Engineering of BUPT. She has authored or coauthored more than 100 papers in international journals and conferences. Her current research interests include resource management and intelligent networking in next generation networks, the Internet of Things, and cloud computing. She was a TPC Member of IEEE WCNC 2012/2014/2015/2016/2019/2020/2021, PIMRC 2012/2017/2018/2019/2020, GLOBECOM 2015/2017/2018, ICC 2015/2016/2017/2018/2019, Infocom 2018, and CloudCom 2013/2014/2015, the Chair of Special Track on cognitive testbed in CHINACOM 2011, the Workshop Chair of IEEE GreenCom 2019, and a Peer Reviewer of many academic journals.

Hong Ji (Senior Member, IEEE) received the B.S. degree in communications engineering, and the M.S. and Ph.D. degrees in information and communications engineering from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 1989, 1992, and 2002, respectively. In 2006, she was a Visiting Scholar with The University of British Columbia, Vancouver, BC, Canada. She is currently a Professor with BUPT. She has authored more than 300 journal/conference papers. Several of her papers had been selected for Best paper. Her research interests include wireless networks and mobile systems, including cloud computing, machine learning, intelligent networks, green communications, radio access, ICT applications, system architectures, management algorithms, and performance evaluations. She has was the Guest Editor of *International Journal of Communication Systems, (Wiley)* Special Issue on Mobile Internet: Content, Security and Terminal. She was the Co-Chair for Chinacom'11, and a Member of the Technical Program Committee of WCNC, Globecom, ISCIT, CITS, WCSP, ICC, ICCC, PIMRC, IEEE VTC, and Mobi-World. She was on the Editorial Boards of the IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING and *Wiley International Journal of Communication Systems*.

Victor C.M. Leung (Life Fellow, IEEE) is currently a Distinguished Professor of computer science and software engineering with Shenzhen University, Shenzhen, China. He is also an Emeritus Professor of electrical and computer engineering and the Director of the Laboratory for Wireless Networks and Mobile Systems of the University of British Columbia (UBC), Vancouver, BC, Canada. He has authored or coauthored on the areas of his research interests which include the broad areas of wireless networks and mobile systems. Dr. Leung was the recipient of the 1977 APEBC Gold Medal, 1977-1981 NSERC Postgraduate Scholarships, IEEE Vancouver Section Centennial Award, 2011 UBC Killam Research Prize, 2017 Canadian Award for Telecommunications Research, 2018 IEEE TCGCC Distinguished Technical Achievement Recognition Award, and 2018 ACM MSWiM Reginald Fessenden Award. He coauthored papers that won the 2017 IEEE ComSoc Fred W. Ellersick Prize, 2017 IEEE Systems Journal Best Paper Award, 2018 IEEE CSIM Best Journal Paper Award, and 2019 IEEE TCGCC Best Journal Paper Award. He was on the Editorial Boards of IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON COMPUTATIONAL SOCIAL SYSTEMS, IEEE ACCESS, IEEE NETWORK, and several other journals. He is a Life Fellow of IEEE, and a Fellow of the Royal Society of Canada (Academy of Science), Canadian Academy of Engineering, and Engineering Institute of Canada. He is named in the current Clarivate Analytics list of "Highly Cited Researchers".